The last few lines will then become:

```
%% Page length
{pagelen, 5},
{mod_signup, [{request_confirm, false}]}
].
```

You will now need to reload the modules from the Zotonic shell:

```
(zotonic001@myhost)2> z:m().
```

You can directly go to pages */signup*, */logon* or */logoff* and try them. Now, you are ready to add them to the menu on the *navbar*. As an administrator, choose the option *Content -> Menu*. Now, go through the following steps:

1. Add menu item.
2. Choose 'New Page'. Select the name 'Signup' and, for convenience, select 'Page menu' as the category. Select the 'Published' option. Then 'Make' the page.
3. Edit the *Signup* menu option.
4. Click on 'Visit full editpage'.
5. In 'Advanced settings', give the page path as'/signup' and the unique name as 'signup' (same as in the dispatch rule). Save your choices.

Similarly, you can add the menu options for logging on and off. These options would then be visible on the navigation bar. Caution: The signup and logon options are silently ignored if you are already signed in.

You can experiment by creating users and signing in from different browsers.

Changing the chat code

You would have noticed that a user signed in on the home page is not visible to the user signed in to the */chat page*. View the *mod_chat/mod_chat.erl* file. Each submission of a chat message is handled by the event function.

The module also implements the behaviour of the *gen_server* module (see *'erl -man gen_server'* for details). This means that the module implements functions like *handle_call, handle_cast*, etc, which are implicitly called by the server.

The event function receives the chat message and notifies the Zotonic server, which will call *handle_cast* with appropriate parameters. Viewing the extract from the *handle_cast* function, you will find that the messages are sent to the list *ChatBoxes2*, which is the list of users signed into chat from the same room. The relevant code lines are:

```
handle_cast({{chat_done, Chat}, Ctx}, State) ->
    case State#state.chat_boxes of
        [] -> nop,
              {noreply, State};
        ChatBoxes ->
              ChatBoxes1 = [ X || X <- ChatBoxes, check_
process_alive(X#chat_box.pid, Ctx)],
              [ChatBox] = [ X || X <- ChatBoxes1, X#chat_box.
pid == Ctx#context.page_pid ],
              ChatBoxes2 = [ X || X <- ChatBoxes1, X#chat_box.
room=:=ChatBox#chat_box.room],
...
                    [F(P) || #chat_box{pid=P} <- ChatBoxes2]
```

Delete the definition of ChatBoxes2 and replace the last line by

```
[F(P) || #chat_box{pid=P} <- ChatBoxes1]
```

In addition, first time a new chat window is displayed, the function handle_call is executed. Relevant function is:

```
handle_call({{add_chat_box, ChatBox}, Ctx}, _From, State) ->
    ChatBoxes = [ X || X <- [ChatBox|State#state.chat_boxes],
check_process_alive(X#chat_box.pid, Ctx)],
    ChatBoxes1 = [ X || X <- ChatBoxes, X#chat_box.room=:=z_
context:get_req_path(Ctx)],
    %% add new chat box to all
    render_userlist_row([{name, ChatBox#chat_box.name},
{upid, format_pid_for_html(ChatBox#chat_box.pid)}],
ChatBoxes1, Ctx),
    %% add existing chat boxes to new
    [ render_userlist_row([{name, CBox#chat_box.name}, {upid,
format_pid_for_html(CBox#chat_box.pid)}], [ChatBox], Ctx) ||
CBox <- tl(ChatBoxes1) ],
    {reply, ok, State#state{chat_boxes=ChatBoxes}};
```

In this case, ChatBoxes is the list of all chat sessions that are still alive and ChatBoxes1 is the subset of chat sessions that are on the same room. You can delete the definition of ChatBoxes1 and replace the references to it by ChatBoxes.

Now, run *z:m()* from the Zotonic shell. All chat sessions should now be visible.

This was just a simple change to illustrate the possibilities. A useful exercise would be to change the subject of chat in each room, with each user entering a particular room to chat with others in that room on a particular topic. Try it as an exercise.

The nice thing is that there was no downtime in all these experiments—not once was a restart of Zotonic needed. END

By: Anil Seth

The author has earned the right to do what interests him. You can find him online at *http://sethanil.com*, *http://sethanil.blogspot.com*, and reach him via email at *anil@sethanil.com*